



Capítulo 9 - Controle de velocidade

Aumentar e reduzir o comando de tração gradualmente, distinguindo o valor solicitado da velocidade real do trem.

Introdução	Até aqui, utilizamos comandos de posição e indicações digitais. Agora vamos estudar um comando variável, que pode aumentar ou diminuir em pequenos passos.
Objetivo	Compreender o princípio do PWM, criar rampas de aceleração e desaceleração e tratar uma solicitação de parada e um bloqueio.
Meta de aprendizagem	Ao final, você deverá conseguir simular um comando de 0 a 100%, limitar o valor solicitado e observar sua variação gradual, sem bloquear o programa.

1. Comando de tração e velocidade real

Em uma locomotiva analógica, a alimentação do motor influencia o movimento. Porém, um mesmo comando pode produzir velocidades diferentes conforme a locomotiva, a carga, a inclinação e as condições mecânicas.

Por isso, o percentual utilizado neste capítulo representa um comando de saída, não uma velocidade medida em quilômetros por hora ou centímetros por segundo.

Comando de 40% não significa 40% da velocidade máxima. A locomotiva pode ainda estar parada, começar a se mover ou desenvolver uma velocidade que depende do conjunto.

Para controlar uma velocidade medida, seria necessário obter informações de deslocamento ao longo do tempo e ajustar a saída com base nessa medição. Esse retorno não está incluído no exercício.

2. O princípio do PWM

PWM significa modulação por largura de pulso. A saída alterna entre ligada e desligada, e o programa altera a proporção do tempo em que ela permanece ligada.

Essa proporção é chamada ciclo de trabalho. Com frequência fixa, aumentar o ciclo de trabalho aumenta o tempo ligado em cada período.

Comando	Comportamento ideal da saída PWM
0%	Permanece desligada.
25%	Fica ligada durante aproximadamente um quarto do período.



Comando	Comportamento ideal da saída PWM
50%	Fica ligada durante aproximadamente metade do período.
100%	Permanece ligada.

Um GPIO gera um sinal de controle de baixa potência. Ele não pode alimentar diretamente o motor da locomotiva nem os trilhos.

3. Onde entra o circuito de potência?

Para tração analógica, um circuito de potência compatível recebe o comando e controla a energia fornecida ao motor. Ele precisa ser adequado à tensão, à corrente, ao sentido de circulação e às proteções do sistema.

Parte do sistema	Responsabilidade
ESP32	Calcular o comando e gerar o sinal lógico.
Interface ou controlador compatível	Receber o sinal sem exceder os limites elétricos da ESP32.
Estágio de potência	Controlar a alimentação da tração.
Proteções e comando de emergência	Atuar conforme o projeto elétrico da ferrovia.

Na Ferrovia EFMN 74, a tração analógica está relacionada à gaveta GAR. Este capítulo não modifica sua montagem, a chave de sentido ou a seleção Analógico/Digital da gaveta de controle principal.

Um módulo PWM com potenciômetro não necessariamente aceita um GPIO no lugar desse potenciômetro. A integração depende da documentação e do circuito de entrada do módulo.

No modo DCC, a central envia comandos ao decodificador da locomotiva. O PWM de bancada deste capítulo não deve ser conectado aos trilhos alimentados pela central DCC.

4. Valor solicitado e valor aplicado

Vamos separar duas variáveis: `targetPercent`, que guarda o valor desejado, e `currentPercent`, que guarda o valor atual do comando.

Se o operador solicitar 30% quando o comando está em zero, o programa não salta diretamente para 30. Ele passa por 1, 2, 3 e assim por diante, até alcançar o alvo.

Na redução normal, ocorre o caminho inverso. Se o alvo for zero, o programa diminui o comando gradualmente.

Esse comportamento é chamado rampa. Ele torna a variação do comando progressiva, mas não garante uma aceleração física constante da locomotiva.



5. Limite e bloqueio

No exemplo, o alvo ficará limitado a 60%. Esse limite é apenas didático; não é uma garantia de velocidade segura e não define o ajuste definitivo da ferrovia.

O comando S solicita uma redução gradual até zero. O comando E interrompe a rampa, zera o comando imediatamente e mantém o programa bloqueado.

O comando R retira o bloqueio e deixa o alvo em zero. Não há retorno automático ao valor anterior: o operador precisa fazer uma nova solicitação.

Zerar o comando elétrico não significa parar instantaneamente um trem. A inércia, a inclinação, o circuito de potência e o modo de alimentação influenciam a parada. O bloqueio de software do exercício não substitui a emergência física da ferrovia.

6. Simulação e indicação por LED

O primeiro teste será feito somente no Monitor Serial. Depois, um LED poderá representar visualmente a saída PWM: seu brilho aumenta ou diminui conforme o comando.

Para o LED de bancada, ligue GPIO 25 → resistor de 1 kΩ → anodo do LED; ligue o catodo ao GND. Faça as conexões com a alimentação desligada.

Esse GPIO é um exemplo para uma ESP32 clássica baseada no ESP32-WROOM. O circuito contém apenas um LED indicador, sem motor, trilhos ou módulo de tração.

O código utiliza `ledcAttach()` e `ledcWrite()`, da API Arduino-ESP32 3.x. Projetos que utilizam o suporte 2.x exigem adaptação da API ou atualização do ambiente antes deste exercício.

O PWM será configurado com frequência de 1.000 Hz e resolução de 8 bits, produzindo valores de 0 a 255. Essa frequência foi escolhida para o LED e não estabelece a frequência adequada para um motor ferroviário.

7. Comandos pelo Monitor Serial

Comando	Ação
+	Aumentar o alvo em 10 pontos percentuais, respeitando o limite.
-	Reduzir o alvo em 10 pontos percentuais, até zero.
S	Solicitar parada gradual.
E	Zerar imediatamente o comando e bloquear novas solicitações.
R	Retirar o bloqueio, mantendo comando e alvo em zero.



Envie um caractere por vez. O programa ignora os caracteres de final de linha. Durante o bloqueio, +, - e S não alteram o alvo.

8. Programa completo

Crie `intermediate_09_speed_control`. Mantenha `simulationOnly = true` no primeiro teste. Copie todos os blocos abaixo, na ordem, para o mesmo programa.

```
const bool simulationOnly = true;
const int ledPin = 25;
const int maxPercent = 60;
const unsigned long rampInterval = 50;
int targetPercent = 0;
int currentPercent = 0;
bool blocked = false;
bool outputReady = false;
unsigned long previousStep = 0;

void applyOutput() {
    int duty = currentPercent * 255 / 100;
    if (!simulationOnly && outputReady) {
        ledcWrite(ledPin, duty);
    }
    Serial.print("Applied command: ");
    Serial.print(currentPercent);
    Serial.println("%");
}

void setTarget(int requestedPercent) {
    if (blocked) {
        Serial.println("Blocked - target unchanged");
        return;
    }
    targetPercent = constrain(requestedPercent, 0, maxPercent);
    Serial.print("Target: ");
    Serial.print(targetPercent);
    Serial.println("%");
}

void stopAndBlock() {
    blocked = true;
    targetPercent = 0;
    currentPercent = 0;
    previousStep = millis();
    applyOutput();
    Serial.println("Blocked - check before restoring");
}

void updateRamp() {
    if (blocked || currentPercent == targetPercent) return;
    unsigned long now = millis();
    if (now - previousStep >= rampInterval) {
        previousStep = now;
        if (currentPercent < targetPercent) currentPercent++;
        else currentPercent--;
        applyOutput();
    }
}
```



```
void setup() {
  Serial.begin(115200);
  delay(1000);
  if (!simulationOnly) {
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);
    outputReady = ledcAttach(ledPin, 1000, 8);
    if (!outputReady) {
      blocked = true;
      Serial.println("PWM setup failed - check configuration");
    }
  }
  Serial.println("EFMN 74 - Gradual command test");
  applyOutput();
  previousStep = millis();
  Serial.println("Commands: + - S E R");
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == '+') setTarget(targetPercent + 10);
    else if (command == '-') setTarget(targetPercent - 10);
    else if (command == 'S') setTarget(0);
    else if (command == 'E') stopAndBlock();
    else if (command == 'R') {
      if (simulationOnly || outputReady) {
        blocked = false;
        targetPercent = 0;
        currentPercent = 0;
        previousStep = millis();
        applyOutput();
        Serial.println("Restored - new target required");
      }
    } else if (command != '\n' && command != '\r') {
      Serial.println("Invalid command");
    }
  }
  updateRamp();
}
```

9. Entendendo a rampa

constrain() limita o alvo entre zero e maxPercent. A função updateRamp() altera o comando em um ponto percentual a cada intervalo de pelo menos 50 ms.

Uma mudança de zero para 60% leva aproximadamente 3 segundos se o programa conseguir manter esse ritmo. Outras tarefas podem alongar o tempo. Não há uma sequência de espera bloqueante dentro da rampa.

A conversão $\text{currentPercent} * 255 / 100$ transforma o percentual em um valor PWM de 8 bits. O resultado é inteiro e pode ter arredondamento para baixo.

outputReady guarda o resultado da configuração da saída PWM. Se a configuração falhar, o programa fica bloqueado; R não habilita uma saída que não foi configurada.

A função applyOutput() apresenta o comando calculado. Na simulação, nenhuma saída PWM é configurada. No teste físico, ela controla somente o LED do exemplo.



Exercício prático - Simulação

Meta: observar alvo, rampa, limite e recuperação do bloqueio.

1. Conecte a ESP32 de bancada por USB, selecione placa e porta e mantenha a simulação ativada.
2. Compile e envie o programa. Abra o Monitor Serial em 115200 baud e pressione EN ou RESET.
3. Confira o comando inicial de 0%.
4. Envie +. Observe o alvo de 10% e os passos até alcançá-lo.
5. Envie + duas vezes, separadamente. Confira o alvo de 30%.
6. Envie -. O alvo deve baixar para 20%, com redução gradual do comando.
7. Envie S. O comando deverá diminuir até zero.
8. Envie + repetidamente até atingir o limite de 60%. Novas solicitações não devem ultrapassar esse limite.
9. Durante uma rampa, envie E. Confira o comando imediato de zero e o bloqueio.
10. Envie + enquanto bloqueado. Confira que o alvo não muda.
11. Após simular a conferência do sistema, envie R. O comando deve permanecer em zero.
12. Envie + novamente para iniciar uma nova solicitação.

Exemplo ao enviar + a partir de zero:

```
Target: 10%  
Applied command: 1%  
Applied command: 2%  
...  
Applied command: 10%
```

Os três pontos representam as linhas intermediárias de 3% a 9%; não são uma mensagem emitida pelo programa.

Teste com o LED

1. Desligue a alimentação e monte o LED com seu resistor.
2. Confira o GPIO e a polaridade.
3. Troque simulationOnly para false, compile e envie.
4. Repita os comandos e observe o brilho variar.
5. Confira que E apaga o LED e que R não recupera o brilho anterior.

O brilho do LED é uma indicação visual do comando, não uma medição de velocidade. A percepção do brilho também não varia necessariamente de forma linear.



10. Aplicação futura à tração

Antes de integrar a ESP32 à gaveta GAR, será necessário conferir o controlador de tração instalado, sua entrada de comando, isolamento ou referência comum, níveis elétricos e comportamento em falhas.

A calibração real deverá identificar o início de movimento, o comando máximo adequado para cada locomotiva e os tempos de rampa compatíveis com o percurso e a composição.

Uma inversão de sentido exige primeiro reduzir a tração a zero e confirmar a parada física antes de alterar o sentido. O exemplo não comanda essa inversão.

No modo DCC, a velocidade e a aceleração dependem dos comandos da central e das configurações do decodificador. A saída PWM deste exercício não representa uma interface DCC.

Não conecte o GPIO 25 aos trilhos, ao motor ou a uma entrada de módulo desconhecida. O único circuito físico especificado neste capítulo é o LED de bancada.

Resumo do capítulo

- Comando de tração e velocidade medida são informações diferentes.
- PWM altera a proporção de tempo ligado da saída.
- A ESP32 precisa de um circuito de potência compatível para controlar um motor.
- Alvo e comando atual permitem criar uma rampa.
- `millis()` permite variar a saída sem bloquear o acompanhamento.
- A parada normal reduz o comando gradualmente.
- O bloqueio zera o comando e exige nova solicitação após a recuperação.
- Zerar a saída não garante parada física instantânea.

Próximo capítulo: Introdução às rotas ferroviárias.

Referência técnica

Espressif: API LEDC do Arduino-ESP32 e guia de migração 2.x para 3.0.
<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html>